

School of Computer Science and Artificial Intelligence

Lab Assignment # 1.5

Program	: B. Tech (CSE)
Specialization	: --
Course Title	: AI Assisted coding
Course Code	:
Semester	: II
Academic Session	: 2025-2026
Name of Student	: A Vamshi Krishna
Enrollment No.	: 2403A51L47
Batch No.	: 52
Date	:11-01-2026

Task 1: AI-Generated Logic Without Modularization (String Reversal Without Functions)**Prompt:** String Reversal Without Functions

```
input_string = "Hello, World!"
reversed_string = ""
for i in range(len(input_string) - 1, -1, -1):
    reversed_string += input_string[i]
print("Reversed String:", reversed_string)
```

```
C:\Users\atur\OneDrive\Desktop\Ai_assistanr>C:/
Reversed String: !dlrow ,olleH
```

Explanation:

- The code takes an input string and initializes an empty string called reversed_string'.
- It then uses a for loop to iterate through the input string in reverse order, appending each character to 'reversed_string'.
- Finally, it prints the reversed string. This approach does not use any functions and directly manipulates the string within the loop.

Task 2: Efficiency & Logic Optimization (Readability Improvement)**Prompt :** Reverse the string 'Hello, World!' without using functions

```
input_string = "Hello, World!"
reversed_string = input_string[::-1]
print("Reversed String:", reversed_string)
```

```
C:\Users\atur\OneDrive\Desktop\Ai_assistanr>C:/
Reversed String: !dlrow ,olleH
```

Explanation:

- The optimized version uses Python's slicing syntax `[::-1]` to reverse the string.
- This approach is more concise and readable than manually iterating through the string.

Task 3: Modular Design Using AI Assistance (String Reversal Using Functions)

Prompt : String Reversal Using Functions

```
def reverse_string(input_string):  
    return input_string[::-1]  
input_string = "Hello, World!"  
reversed_string = reverse_string(input_string)  
print("Reversed String:", reversed_string)
```

```
C:\Users\atur\OneDrive\Desktop\Ai_assistanr>C:/  
Reversed String: !dlrow ,olleH
```

Explanation:

- The function `reverse\_string` takes an input string and returns its reversed version using slicing.
- This modular approach improves code readability and reusability, allowing us to easily reverse any string by calling the function.

Task 5: AI-Generated Iterative vs Recursive Fibonacci Approaches (Different Algorithmic Approaches to String Reversal)

Prompt: Write two different algorithms to reverse a string: one using an iterative approach and another using a recursive approach.

```
def reverse_string_iterative(input_string):  
    reversed_string = ""  
    for i in range(len(input_string) - 1, -1, -1):  
        reversed_string += input_string[i]  
    return reversed_string  
  
def reverse_string_recursive(input_string):  
    if len(input_string) == 0:  
        return ""  
    else:  
        return input_string[-1] + reverse_string_recursive(input_string[:-1])  
input_string = "Hello, World!"  
reversed_iterative = reverse_string_iterative(input_string)  
reversed_recursive = reverse_string_recursive(input_string)  
print("Reversed String (Iterative):", reversed_iterative)  
print("Reversed String (Recursive):", reversed_recursive)
```

```
C:\Users\etun\OneDrive\Desktop\Ai_assistant>C:/Python314/py
Reversed String (Iterative): !dlrow ,olleH
Reversed String (Recursive): !dlrow ,olleH
```

Explanation:

- The iterative approach builds the reversed string by iterating through the input string in reverse order,
- while the recursive approach constructs the reversed string by taking the last character and recursively reversing the rest of the string.
- Both methods achieve the same result, but the recursive approach can be less efficient for long strings due to the overhead of multiple function calls.